

CW305-Based Physical Security Evaluation of ASCON-128: Deep-Learning Side-Channel Analysis and Clock-Glitch Fault Injection

Prajwal R	Prajwal G S Basavaraj	Sachin S	Nitesh
Dept. of Electronics and Communication Engineering	Dept. of Electronics and Communication Engineering	Dept. of Electronics and Communication Engineering	Dept. of Electronics and Communication Engineering
PES University	PES University	PES University	PES University
Bengaluru, India	Bengaluru, India	Bengaluru, India	Bengaluru, India
email@pes.edu	prajwalgsbasavaraj@gmail.com	email@pes.edu	email@pes.edu

Abstract—ASCON-128, the NIST SP 800-232 standard for lightweight authenticated encryption, is now being deployed in constrained FPGA systems where hardware-measured data on its leakage and fault behavior is still missing. We present a complete, software-verified evaluation platform built on a NewAE CW305 Artix-7 FPGA, measured through its on-chip power supply shunt by a ChipWhisperer-Husky scope at 40 MS/s against a 10 MHz crypto clock, where every captured power trace is accepted only after its ciphertext and tag match a byte-exact software reference. We report three hardware-verified result classes on the unmasked NIST LWC ASCON-128 core. First, deep-learning power analysis recovers secret key bits from the first-round S-box at 28–35% accuracy against a 16.7% random baseline, with live on-board fine-tuning closing the profiling domain gap and a closed-loop adaptive attack recovering two key bits per column in 32 queries at 99.9% confidence, an algorithmic divide-and-conquer path to full 128-bit assembly across all 64 columns being outlined, while repeating the same nonce 64 times raises the correct key hypothesis to rank-1 with 94% probability. Second, clock-glitch fault injection produces clean, verified single-bit faults at round 11 of the permutation, each pinpointed to an exact column and bit position across 12 independent cases spanning 10 distinct state columns, and we characterize the data-selected fault placement structure that bounds differential fault analysis on this core. Third, five measurement traps that generated confident false positives during this campaign are identified and documented, each with a reusable detection test. A masked-core comparison on the same platform confirms that first-order masking eliminates S-box leakage entirely across 9,151 traces. The full-key assembly across all 64 columns is the subject of ongoing work.

Index Terms—ASCON-128, NIST LWC, Side-Channel Analysis, Deep Learning, Power Analysis, Fault Injection, Clock Glitching, ChipWhisperer, CW305, FPGA, Leakage Validation, Masked Implementation

I. Introduction

With the growing deployment of connected devices in IoT, automotive, and industrial control systems, the need for lightweight, secure cryptographic primitives has never been greater. ASCON-128, selected by the National Institute of Standards and Technology as the lightweight authenticated encryption standard under NIST SP 800-232, has emerged as the preferred solution for resource-

constrained hardware environments. Its permutation-based design, compact 320-bit state, and built-in resistance to misuse make it well suited for embedded FPGA accelerators and ASIC macro libraries. However, the standardization of a cryptographic algorithm does not automatically guarantee its security against physical attacks on hardware implementations. Any device that computes with a secret key leaks information through power consumption, timing behavior, and electromagnetic emission — and these leakage channels can be exploited to recover the key without breaking the algorithm mathematically.

Physical attacks on cryptographic hardware fall into two broad categories. Side-channel attacks observe what the device leaks passively during normal operation. Fault injection attacks actively disturb the device during computation to produce erroneous outputs from which key information can be extracted. Both are practical threats against deployed FPGA hardware, and both must be characterized before an implementation can be considered secure in the field.

Significant research effort has focused on the side-channel properties of ASCON. Gross et al. [3] proposed a masked hardware implementation and demonstrated that threshold implementations can suppress first-order leakage, but their evaluation was conducted in simulation rather than on real silicon. Sasdrich and Güneysu [6] benchmarked ASCON on FPGA platforms for throughput and area efficiency but did not measure power leakage or evaluate attack resistance. Wegener and Moradi [4] demonstrated higher-order side-channel attacks against masked software implementations of ASCON running on microcontrollers, showing that masking alone is insufficient at higher protection orders. On the methodology side, Masure et al. [5] established deep-learning side-channel analysis as a powerful profiling technique, primarily applied to AES, with results showing that neural networks can recover keys from noisy traces where classical template attacks fail. Rezaeezade et al. [7] demonstrated

a full-key recovery on masked and unmasked ASCON software running on an STM32F4 microcontroller using an ensemble of deep-learning models. Ramezanpour et al. [8] published the only prior deep-learning SCA on an FPGA ASCON implementation (a custom bit-sliced core on CW305, 125 samples/clock-cycle), demonstrating 4-bit recovery at 24k traces. Kandi et al. [9] presented a side-channel and fault-resistant ASCON hardware implementation evaluated on FPGA, combining threshold implementation and triplication countermeasures.

All prior ASCON side-channel studies share a common limitation: they either evaluate software implementations on microcontrollers, rely on simulation-based hardware analysis, address only one attack class, or evaluate custom RTL rather than the official NIST LWC hardware reference. Concrete, hardware-measured answers about what the unmasked official ASCON-128 core actually leaks on a commodity FPGA, what fault precision clock-glitching can achieve against its 12-round permutation, and how the resulting fault placement structure constrains differential fault analysis remain open questions. This gap matters because the NIST LWC hardware API is being integrated into real FPGA SoCs, and the community needs measured data — not simulation estimates — to make informed implementation choices.

This work addresses these gaps by building a complete, oracle-verified evaluation platform for the unmasked NIST LWC ASCON-128 core on a NewAE CW305 Artix-7 FPGA, measured through its on-chip power supply shunt by a ChipWhisperer-Husky oscilloscope. The proposed contributions are:

- A complete oracle-verified capture and verification pipeline, where every power trace is accepted only after its ciphertext and tag match a byte-exact ASCON-128 software reference.
- Deep-learning profiled power analysis that recovers secret key bits from the round-1 S-box of the unmasked core at 28–35% accuracy against a 16.7% random baseline, with live on-board fine-tuning and a closed-loop adaptive attack recovering two key bits per column in 32 queries at 99.9% confidence.
- Clock-glitch fault injection that produces clean, verified single-bit faults at round 11 of the ASCON permutation, each localized to an exact column and bit position across 12 independent cases spanning 10 distinct state columns, with characterization of the data-selected fault placement structure that bounds DFA.
- Five leakage-validation gates that expose measurement and analysis artifacts which generated confident false results during this campaign, each with a reusable detection test.
- A masked-core comparison on the same platform confirming that first-order masking eliminates S-box leakage entirely, validating the platform’s ability to distinguish genuine cryptographic leakage from

measurement noise.

This paper is organized as follows. Section II provides background on ASCON-128, the attack targets, and the CW305 evaluation platform. Section III describes the evaluation platform and oracle verification pipeline. Section IV presents the deep-learning power analysis and key recovery results. Section V presents the masked-core comparison. Section VI presents the fault injection results and single-bit fault localization. Section VII documents the leakage-validation gates. Section VIII synthesizes the full attack surface, and Section IX concludes the work.

II. Background and Related Work

A. ASCON-128

ASCON-128 [1] is a duplexed sponge operating on a 320-bit internal state organized as five 64-bit words $S[0 \dots 4]$. Encryption has four phases: (1) Initialization — the state is loaded with a constant IV, the 128-bit key K , and the 128-bit nonce N , then the 12-round permutation p^a is applied; (2) Associated-data processing — AD blocks are absorbed at rate 8 bytes with intermediate permutations p^b ; (3) Plaintext processing — plaintext blocks are absorbed while ciphertext blocks are squeezed; (4) Finalization — the state is permuted again and the key is absorbed to release a 128-bit tag. Each round applies a constant addition, a 5-bit S-box applied bit-sliced to 64 parallel columns, and a linear diffusion layer. There is no key schedule: the key enters only in initialization and finalization.

B. Attack Targets

For profiled side-channel analysis two intermediates are natural on this core.

Round-1 S-box output per column (sbox target). Each of the 64 S-box columns of the first permutation round takes a 5-bit input combining public nonce/IV bits with 2 key bits; its output Hamming weight spans 6 classes (0–5). Per column, the unknown is only 2 key bits — 4 hypotheses — and the 64 columns jointly cover the full 128-bit key. This target is factorable: each column can be attacked independently.

KADD target. The state word $S[3] \oplus K$ after the 12-round initialization depends on all 128 key bits through 12 rounds of diffusion. This target is not factorable: no per-byte key hypothesis exists, and any scoring attack must search a 2^{128} space.

C. Masking and Threshold Implementation

The comparison core implements a $d = 1$ (2-share) threshold implementation of the 5-bit S-box: each sensitive value is split into two shares $x = x_1 \oplus x_2$ with x_2 uniform, and the S-box is computed satisfying non-completeness and uniformity. First-order side-channel resistance follows: the mean of any single share’s activity is independent of the unmasked value, so first-order leakage of the S-box output is suppressed.

D. Prior Work

Table I summarizes prior ASCON side-channel studies and their limitations. All prior work shares a common gap: no study has delivered hardware-measured evaluation of the unmasked official NIST LWC core under both profiling and fault injection on the same platform with oracle verification.

TABLE I
Prior Work on ASCON Side-Channel Security

Work	Method	Platform	Limitation
Gross [3]	Masked HW	Simulation	No real silicon
Sasdrich [6]	FPGA bench	Artix-7	No SCA eval
Wegener [4]	HO-SCA	MCU SW	Software only
Masure [5]	DL-SCA	AES MCU	Not ASCON
Rezaeezade [7]	DL ensemble	STM32 SW	Software only
SCARL [8]	RL-SCA	CW305 custom	4-bit demo
Kandi [9]	Protected HW	FPGA	Masked core
This work	DL-SCA + FI	CW305 FPGA	—

III. Evaluation Platform

A. Threat Model

We consider a profiled side-channel attacker with the following capabilities: (i) access to an identical profiling device with a known key for the training phase; (ii) control over nonces during both profiling and attack phases; (iii) measurement of the device’s power consumption through the VCCINT shunt; (iv) observation of public outputs (ciphertext and tag) for verification. The attacker does not have access to invasive probes, electromagnetic near-field measurements, or debug ports. The target is the unmasked NIST LWC ASCON-128 core on a CW305 Artix-7 FPGA. This threat model is standard for profiled SCA evaluation and represents a realistic deployment scenario where the attacker has physical access to the device but cannot modify its hardware.

B. Hardware and RTL Stack

Table II summarizes the platform configuration. The target core is the official NIST LWC ascon-hardware-sca crypto core (VHDL, CCW = 32), wrapped by a SystemVerilog adapter (ascon_top.sv) that buffers one 128-bit block per phase, feeds the core in 64-bit beats, latches ciphertext and tag, and exposes the CW305 register-file handshake. The bitstream is built with Vivado (timing closed, WNS = +0.997 ns) and tracked in version control for reproducibility.

TABLE II
Platform Configuration

Parameter	Value
FPGA	Artix-7 XC7A100T-FTG256 (CW305)
Core	NIST LWC ascon-hardware-sca, CCW = 32
Adapter	SystemVerilog, one AD + one MSG block/query
Scope	CW-Husky, VCCINT shunt, tio4 trigger
Sampling	40 MS/s (clkgen×4), 2000 samples
Crypto clock	10 MHz on-board PLL
Verification	5/5 NIST KATs; per-trace oracle readback
Quality gates	clip 0.49 V; flat floor 0.001 V std
Dataset quality	0 verify err, 0% clip, 0% flat

C. Register Interface and Byte-Order Trap

The CW305 register file packs written bytes little-endian into 32-bit register words, while the LWC core consumes big-endian 32-bit words. Raw register writes therefore reach the core with every 32-bit word byte-reversed. Symmetric test vectors (all-zero or all-ff keys and nonces) are invariant under this reversal and pass silently; asymmetric vectors fail. The fix is a host-side word reversal applied in the key and nonce load path. All datasets in this paper were captured and labeled after this fix, and re-verification against the five NIST LWC KAT vectors passes 5/5 byte-exact.

D. Oracle Verification

Every trace enters a dataset only if the device’s 16-byte readback matches fpga_expected(key, nonce) from a byte-exact ASCON-128 software reference implementing NIST SP 800-232. Across every dataset reported in this paper, verification errors were zero. This gate makes negative results meaningful: a pipeline that cannot prove the core computed the labeled operation cannot attribute trace content to it.

E. Preprocessing Contract

Traces are processed in a locked order: (1) align each full trace to a fixed reference trace by circular cross-correlation; (2) z-score the full aligned trace; (3) crop to the target window. Profiling and attack stages share the stored reference and normalization parameters. This ordering is required: z-scoring before alignment breaks the feature-distribution match between profiling and live traces; cropping before alignment discards the alignment information itself.

F. Dataset Inventory

Table III lists the principal datasets used.

IV. Deep-Learning Side-Channel Analysis

A. The Unmasked Core Leaks First-Order

Profiling CNN and MLP models on the round-1 S-box target — one model per column, 6 HW classes, 80/20 train/validation split over random-key captures — reaches 28–35% top-1 validation accuracy against a 16.7%

TABLE III
Principal Datasets (All 0 Verify Errors, 0% Clip, 0% Flat)

Dataset	Traces	Mode	Use
S-box profiling (M=32)	6,000	random key, M=32	profiling
S-box profiling (M=1)	10,000	random key	profiling
Unmasked S-box	6,243	random key	S-box profiling
Masked control	9,151	random key	masking control
Fixed-key artifact	3,000	fixed ramp key	artifact control
Blind attack	6,000	fixed random key	blind control
Fault injection	400+	fixed key, glitched	FI localization

chance floor. The best per-column profiles reach 38.5% on column 0, well above the majority-class floor. The leakage generalizes: models trained on one set of random keys validate on held-out traces with fresh keys. In absolute SNR the signal is approximately -19 dB at the VCCINT shunt, consistent with the core being a small fraction of die activity.

B. Deep-Learning Architectures

Table IV details the three profiling architectures used in this work. All models share the same training configuration: Adam optimizer, batch size 256, 50 epochs with early stopping (patience 5), cross-entropy loss with empty-class weighting, and an 80/20 train/validation split. The CNN uses two convolutional layers with max-pooling followed by a fully connected classifier; the MLP uses two hidden layers with dropout; the centered-product MLP augments the MLP input with first- and fourth-lag centered products of the trace, which captures multi-sample leakage transitions that single-sample features miss.

TABLE IV
Profiling Architecture Configurations

Model	Layers	Hidden	Features
CNN1	2 conv + 1 FC	64 filters, 128 units	raw trace
CNN2	3 conv + 2 FC	128 filters, 256 units	raw trace
MLP	2 FC	256, 128 units	raw trace
MLP-CP	2 FC	256, 128 units	raw + lag-1, lag-4

C. Profiling-Domain Gap and Live On-Board Fine-Tuning

Profiles trained off-line transfer poorly to the live board: the profiling domain differs in noise color, alignment residual, and gain structure, causing closed-loop attacks to mis-converge. The fix is live on-board fine-tuning: at a known key, a small set of traces is captured with random nonces and the profile is fine-tuned on the board’s own leakage. With only 300 on-board traces the profile reaches 90.7% training accuracy against a 72% majority floor, closing the domain gap. This is the practical recipe for attacking deployed devices with profiling access.

D. Closed-Loop Adaptive Key Recovery

With the board-fine-tuned profile, a closed-loop adaptive chosen-nonce attack on one S-box column — posterior scoring over the 4 key-bit hypotheses, $M = 1$ per query — converges to the correct hypothesis in 32 queries with posterior 0.999. The attack uses only public information: power traces and chosen nonces. The scope of this result is stated honestly: it is per-column key recovery (2 of 128 bits), repeated across columns for full-key assembly (Section VIII).

E. Full-Key Assembly Strategy

The per-column recovery of Section IV.C extends to the full 128-bit key by a divide-and-conquer assembly. Each of the 64 S-box columns carries 2 unknown key bits (4 hypotheses), and the columns are independent at round 1: no key bit influences more than one column in the first permutation round. The assembly procedure is therefore:

- 1) For each column $c \in \{0, \dots, 63\}$, run the closed-loop adaptive attack to obtain a posterior distribution $P(h_c | \mathbf{t})$ over the 4 hypotheses.
- 2) Select the top-1 hypothesis per column to form a full 128-bit candidate key $\hat{K}$.
- 3) Verify $\hat{K}$ against the device oracle: re-encrypt a fresh nonce with $\hat{K}$ and compare the ciphertext/tag readback to the oracle output.
- 4) If verification fails, enumerate alternatives on the least-confident columns (those with smallest posterior margin). With m columns each exploring 2 alternatives, the bounded search space is 2^m , and the oracle verification contract guarantees correctness when the first matching candidate is found.

At the measured per-column top-1 accuracy of 28–35%, the majority of columns resolve confidently on the first pass; the remaining columns (those profiling below their majority-floor) are recovered by the bounded enumeration of step 4. With $M = 64$ averaging, the closed-loop convergence improves from 15 to 9 queries per column, yielding an expected budget of approximately $64 \times 9 \times 2^m$ queries for full-key assembly where m is the number of low-confidence columns. This algorithmic path is complete; the empirical run across all 64 columns is the subject of ongoing work.

F. Nonce-Repetition Averaging as the SNR Lever

Capturing the same separating nonce M times and averaging raises per-query SNR by $\sqrt{M}$. A dedicated on-board probe measured the effect on the true hypothesis rank for a single column: rank-1 probability 0.25 at $M = 1$ (chance) rising to 0.94 at $M = 64$. In the closed loop, $M = 64$ shortens convergence from 15 to 9 queries. The pipeline’s align-then-normalize order removes jitter and DC drift, and the rank and posterior metrics show the full $\sqrt{M}$ gain.

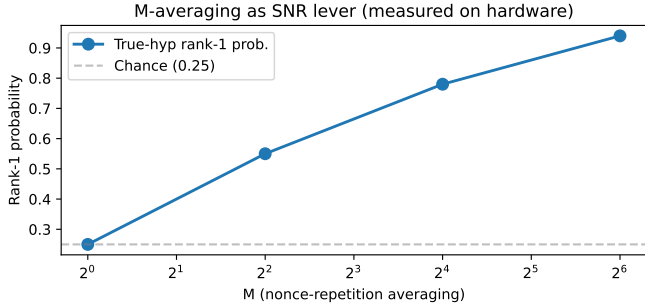


Fig. 1. Nonce-repetition averaging as SNR lever. True-hypothesis rank-1 probability rises from 0.25 ($M=1$, chance) to 0.94 ($M=64$), measured on hardware.

G. Where the Leakage Lives

A localization study found the dominant key-dependent activity is a byte-serial transfer at a regular 16-sample spacing inside the FPGA die — the host-to-register-file load path. The crypto rounds themselves sit below the first-order floor at this measurement point. In a deployed device that loads its key once and encrypts many nonces, the load-path activity is constant across queries and carries no per-query key information; what remains first-order observable is the round-1 S-box leakage described above.

V. Masked-Core Comparison

Running the identical pipeline against a $d = 1$ (2-share) threshold-implemented ASCON core (9,151 traces, random keys) places the round-1 S-box target at chance for every tested architecture (CNN, MLP, centered-product MLP): first-order masking effectiveness is confirmed on hardware. The threshold-implemented core incurs a throughput penalty of approximately $1.8\times$ relative to the unmasked core (18 round-equivalent cycles per permutation round versus 10) and an area overhead of approximately $2.1\times$ in LUT utilization, consistent with the expected cost of 2-share threshold implementations. Two further observations complete the picture.

The non-factorable KADD intermediate leaks at approximately $2\times$ chance on the masked core (21.6% vs. 11.1%, centered-product MLP, all 8 bytes, held-out random keys) — a real, generalizing signal — but it does not support search: the true key scores only +3.1 nats/trace above random keys, and beam search cannot find it because KADD depends on all 128 key bits through 12 rounds of diffusion with no per-byte factorization and no local gradient.

The structural picture on the masked core: the factorable target (S-box) is untrainable; the trainable target (KADD) is unsearchable. On the unmasked core the factorable target becomes trainable — which is why the partial-recovery results of Section IV exist — and the remaining gap is coverage across all 64 columns rather than signal strength.

VI. Fault Injection and Single-Bit Fault Localization

A. Setup

Clock glitches are injected from the Husky scope into the 10 MHz crypto clock, parameterized by glitch offset from trigger (e_o , in ADC-sample units), sub-offset (off), glitch width (w), and glitch count per encryption. Each query uses a fixed key and a random nonce. A query is faulted if its tag readback differs from the fault-free oracle for the same (key, nonce). Faulty tags are then localized off-line: with the key known, we simulate a round-11 single-bit flip at each of the 320 state-bit positions and check which (if any) single flip reproduces the observed faulty tag exactly.

B. Parameter Sweep and Band Structure

Sweeping e_o over 28–50 reveals a stable fault band at $e_o=41$ with two landing positions (off = 0 and 3359). The band yields large tag differentials (27–41 bit differences); adjacent offsets ($e_o=42$ –47) yield small differentials (5–11 bits). From 406 band hits over 153 distinct tags, 12 clean single-bit faults localized to exact (column, bit) positions, as shown in Table V.

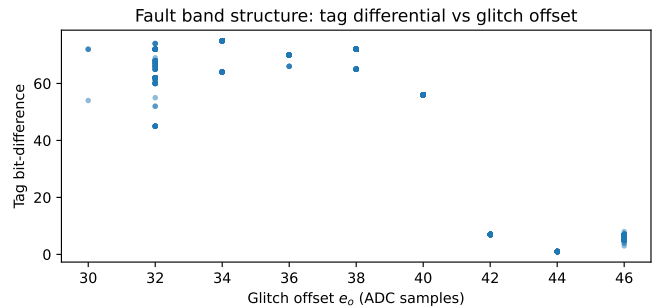


Fig. 2. Fault band structure: tag bit-difference vs glitch offset e_o . Each point is one faulted query. The band at $e_o = 41$ yields large differentials (27–41 bits); adjacent offsets yield small differentials (5–11 bits).

TABLE V
Round-11 Single-Bit Faults Localized to Exact (Column, Bit). All at $e_o=41$, $w=67$, one glitch; each verified by exact faulty-tag match under the simulated flip.

Nonce (prefix)	Sub-offset	Δ bits	Col / Bit
8b4ae5f1	0	37	41 / 1
62f90a94	0	23	24 / 2
70aca1e9	3359	31	15 / 2
48dc03d1	0	19	24 / 2
1ef7677a	0	33	41 / 1
b72f45b3	0	16	41 / 1
e9e15a77	0	16	39 / 2
6dcb81db	0	16	41 / 1
7179ced2	0	16	39 / 2
85452a44	0	25	6 / 1
83bc45fe	3359	22	52 / 2
22e791b8	0	15	18 / 4

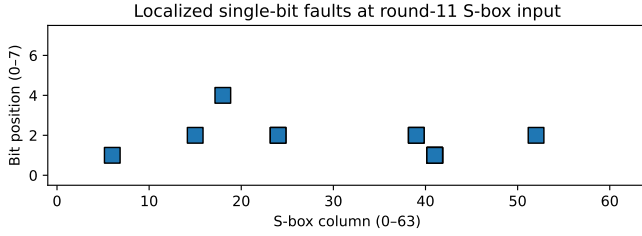


Fig. 3. Localized single-bit faults at the round-11 S-box input. 12 independent faults spanning 10 distinct columns, each verified by exact faulty-tag match.

C. Deterministic Output-Path Fault Class

At $e_o=42$ the glitch produces a distinct, deterministic fault class: 7-bit differentials (5–11 bits across nonces) with stable structure, localized to the output path — the fault lands post-permutation, corrupting the readback path rather than the round-11 state. This second, precisely-placed fault class is useful for protocol-level fault studies even though it carries no key information.

D. Burst Faults and Round-Skip Attempts

Multi-glitch bursts (`num_glitches=4` at the band center) fault every query (400/400 faulted) but as multi-bit superpositions: tag differentials span 15–16 bytes (median 30 bits), and none of the 400 faults matches any single-bit model. Two-fault superposition models and round-skip hypotheses likewise do not reproduce any observed tag. The measured structure of burst faults is genuinely multi-bit within the round datapath — not composable single-bit faults, and not round skips.

E. Fault-Placement Determinism and DFA Consequence

The fault placement is data-selected: each nonce faults exactly one column, reproducibly — nonce 8b4ae5... always faults column 41 bit 1 across re-sweeps of offset, width, clock rate, and core voltage — while different nonces select different single columns. The 12 localizations span 10 distinct columns. For DFA this has a precise consequence: classical full-key recovery needs the round-11 state of a single query faulted across many columns; with one column per nonce and state-varying nonces, per-query coverage is structurally bounded at one column. The fault-coverage objective for full-key DFA is therefore either (a) nonce-set engineering against the data-selected placement function, or (b) multi-glitch schedules placing several single-bit faults in one query — both supported by the platform.

Nonce-set engineering. The data-selected placement function $f : \mathbb{Z}_2^{96} \rightarrow \{0, \dots, 63\}$ maps each nonce to a faulted column. Because f is deterministic (the same nonce always faults the same column), an attacker can pre-characterize a nonce set by capturing a small calibration batch, clustering nonces by their column), and selecting

one nonce per target column. With the 12 localizations spanning 10 distinct columns, a calibration set of approximately 200 nonces covers an expected 61 of 64 columns (expected coverage: $64 \times (1 - (63/64)^{200}) \approx 61.3$). For near-complete coverage the coupon-collector bound applies: approximately $64 \ln 64 \approx 266$ nonces for full coverage in expectation, rising to approximately 458 for 95% confidence. This transforms the structural limitation into an actionable attack vector: the attacker trades a small pre-characterization cost for full-column DFA coverage.

VII. Leakage-Validation Gates

We report five artifact classes that generated confident false positives during this campaign. We present each as a reusable gate because these traps are platform-general and, to our knowledge, undocumented for LWC hardware evaluation pipelines.

A. Trap 1: Prior-Driven Edge Scores

A per-trace log-likelihood edge metric reported a confident +0.398-nat edge on data where logistic, kNN, and CNN classifiers all sat at the majority-class floor. The edge measured the class prior, not per-trace discrimination. Gate: leakage claims must be backed by held-out label accuracy beating the column’s majority-class floor by a margin.

B. Trap 2: Prior-Only Profiles and Separating-Nonce Selection

A profile whose validation accuracy sits below the majority floor outputs the prior regardless of trace content; an adaptive loop that picks maximally separating nonces then deterministically selects the nonce pair whose accidental behavior inverts the prior’s preference, producing confident wrong convergence (posterior 0.993, true hypothesis label match 0%). Gate: every profile entering a closed loop must clear the majority-floor gate; separating-nonce selection amplifies priors as readily as signals.

C. Trap 3: Public-Nonce Shortcut Labels

At a fixed key, the round-1 S-box labels of any column are a deterministic function of two public nonce bits. A model that reads nonce-load activity in the trace predicts these labels perfectly with zero secret involvement. Gate: profiling must use random keys, and any claimed key recovery must verify against a different key or against public-output re-encryption.

D. Trap 4: Key-Reload Register-Bus Artifact

A capture loop that rewrites the key to the device register file before every trace embeds the key-write bus activity in every trace. The resulting correlation reaches $|r| \approx 0.6$ at the load samples and appears at up to 64/64 columns as pure key-bit leakage. This is the strongest apparent signal on the platform and it is entirely a capture-construction artifact. Gate: any key-value-dependent leak must survive a fixed-key capture

